Speaker Script

Tiny Tapeout Workshop Series

Northwestern IEEE · Open Silicon 2026

Slide-aligned — one entry per slide, in deck order.

For the session lead on stage. Helper troubleshooting lives in `moderator_troubleshooting.pdf`.

Auto-generated from the `\note{}` blocks in the slide decks.
Edit a slide → rebuild this script: `python3 build_speaker_script.py`.

Contents

Beginner Session **4**
1. Welcome . . . 4
2. About Me . . . 4
3. What You Can Learn at Northwestern . . . 4
4. What This Lab Is / Is Not . . . 5
5. Open Source Is Coming for Chip Design . . . 5
6. But the Commercial Stack Still Exists . . . 6
7. And You Can Still Take It Anywhere . . . 7
8. Today's Plan . . . 8
9. What Is Digital Communication? . . . 8
10. From Logic to Computation . . . 8
11. The Transistor Revolution . . . 8
12. How a Transistor Works . . . 9
13. How Is a Transistor Actually Made? . . . 9
14. Why Does This Matter for Us? . . . 9
15. The Secret Sauce: PDKs . . . 10
16. From Open PDK to Open Fabrication . . . 10
17. Tiny Tapeout: Chip Design for Everyone . . . 10
18. What You Need . . . 10
19. Level 1: Draw a Transistor in SiliWiz . . . 10
20. Level 1: What You Just Built . . . 11
21. How a NAND Gate Is Built on Silicon . . . 11
22. Level 2: NAND Gate — Truth Table . . . 11
23. Level 2: The Flip-Flop — Adding Memory . . . 11
24. Level 2: The Full Adder — How Fast It Scales . . . 11
25. Level 2: Build Your Own Gate in Wokwi . . . 12
26. Level 3: Fork the Template . . . 12
27. Level 3: Link Your Wokwi Design . . . 12
28. Level 3: Watch the Action . . . 12
29. Level 3: View in 3D . . . 13
30. From Truth Table to Gates — Automatically . . . 13
31. OK — Let's Talk Verilog . . . 13
32. Level 4: What Is This Code? . . . 13
33. Level 4: Two Kinds of Verilog . . . 14
34. Level 4 Goal: Reverse the Stripes . . . 14
35. Templates and Module Naming . . . 14
36. Level 4 Goal: GDS on GitHub Actions . . . 14
37. Level 4: Submit . . . 15
38. How Submission Actually Works . . . 15
39. Your Tile's Interface: 26 Pins . . . 15
40. A Note on IP and Open Source . . . 15
41. Testing Your Design . . . 15
42. Tapeout Assignments — Let's Talk . . . 16
43. Level 5: Install OSS CAD Suite . . . 16
44. Level 5: What Is OSS CAD Suite? . . . 16
45. Level 5: FPGA → ASIC — What to Fix . . . 16
46. Level 5: Simulate and Synthesize . . . 16
47. Level 5: Get Into Tiny Tapeout . . . 17

48. Key Deadlines . . . 17

Advanced Session **18**
1. Tiny Tapeout: Advanced (Analog Design) . . . 18
2. About Me . . . 18
3. What You Can Learn at Northwestern . . . 18
4. What This Lab Is / Is Not . . . 18
5. Open Source Is Coming for Chip Design . . . 19
6. But the Commercial Stack Still Exists . . . 19
7. The Session Arc . . . 19
8. TT Tile Interface: The HDL Contract . . . 19
9. Part 1: VGA Playground — Verilog in Practice . . . 20
10. Part 2: Install OSS CAD Suite . . . 20
11. Part 2: OSS CAD Suite vs. Quartus / Synplify . . . 20
12. Part 2: The Local Workflow . . . 20
13. Part 2: What `stat` Tells You . . . 20
14. Part 2: From Tiny Tapeout to Production . . . 21
15. Tapeout Requirements — Let's Talk . . . 21
16. Part 3: The Mixed-Signal Reveal . . . 21
17. Part 3: What This Means for sky130 . . . 22
18. The File Ecosystem: Design Files . . . 22
19. The File Ecosystem: Output and Submission . . . 22
20. Timing, Models, and Characterization . . . 22
21. Analog Tile Specifications . . . 22
22. Analog Submission Requirements . . . 23
23. How to Purchase and Submit . . . 23
24. IP and Legal Considerations . . . 23
25. Is Tiny Tapeout Right for Your Lab? . . . 23
26. Part 4: Setup iic-osic-tools . . . 24
27. Part 4: Create a Project . . . 24
28. Part 4: Place Transistors . . . 24
29. Part 4: Add Pins . . . 24
30. Part 4: Wire the Inverter . . . 24
31. Part 4: Set Transistor Parameters . . . 24
32. Part 4: Create a Symbol . . . 24
33. Part 4: Build the Testbench . . . 25
34. Part 4: Configure Component Values . . . 25
35. Part 4: Add Simulation Commands . . . 25
36. Part 4: Run Simulation . . . 25
37. Part 4: Add Measurements . . . 25
38. Part 5: Key sky130 Layers . . . 25
39. Part 5: Draw the Layout . . . 26
40. Part 5: DRC . . . 26
41. Part 5: LVS . . . 26
42. Part 6: Parasitic Extraction . . . 26
43. Part 6: Run Post-Layout Simulation . . . 26
44. Part 6: Compare Results . . . 27
45. Part 7: Export from Magic . . . 27
46. Part 7: Use the sky130 Analog Template . . . 27
47. Key Deadlines . . . 27
48. XSCHEM Shortcuts Reference . . . 27

TA Session **28**

1. Tiny Tapeout: TA Session 28
2. While I Talk — Get xschem Running 28
3. About Me 28
4. What This Lab Is / Is Not 28
5. The Plan 28
6. Two Tiles, Not Two Designs 29

Beginner Session

Friday April 17, 2026

Slide 1 Welcome

Welcome to Tiny Tapeout! I'm Jack Thoene, ECE PhD here at NU.

- Tonight we are going to design a real silicon chip.
- Not a simulation. Not a model. An actual chip that will be fabricated and mailed back to you.
- Before we start, make sure you have:
  - Laptop plugged in
  - GitHub account (github.com — make one now if you don't have one)
  - Chrome or Firefox open
- Everything is browser-based. No installs. No setup.

Slide 2 About Me

Quick background.

- VAK Embodied System Lab with Prof. Arora
- Ultra-low-power, battery-free sensors, energy-harvesting IoT — runs on microwatts
- **The problem:** best COTS low-power chip = Cortex-M0 . That's the floor.
- For plant sensors, wearable patches, environmental monitors — even an M0 is too much
- Can we design our own silicon, optimized for exactly what we need?
- That's a *systems* problem, not a VLSI-lab problem
- Co-design materials + hardware + software
- Tiny Tapeout lets systems people like me actually tape out a chip.

Slide 3 What You Can Learn at Northwestern

- NU has courses for nearly every step — logic, devices, fab, analog, comp arch, micro/nano fab
- You can learn everything on this list. Except one thing: actually taping out a chip.
- For that — you have to join a VLSI lab

- **But what if you're not a VLSI person?**
- Skills, licenses, months sunk into one design
- Chip design is still a destination, not a tool.
- Inaccessible unless it's the only thing you do
- Show of hands — who here has never touched digital design? Analog?
- That's exactly who this workshop is for. Tonight we change that.

Slide 4 What This Lab Is / Is Not

Set expectations.

- **IS:** hands-on taste of the open-source chip design flow
- Every step has written instructions — no prereqs
- **IS NOT:** a full VLSI course
- Not something you need to finish tonight. Definitely not graded.
- Go at your own pace, ask questions, break things. That's how you learn.

Slide 5 Open Source Is Coming for Chip Design

Two parallel timelines. One already happened.

- **KiCad arc:**
  - 2004 — French university toy. Nobody serious touched it.
  - 2013 — CERN adopts it, hires engineers, contributes a real router
  - 2018 — Adafruit, Arduino, Olimex publish boards in KiCad
  - 2024 — SpaceX, Google, Meta, Framework all have it in their flow
- Twenty years ago, telling a PCB engineer they'd design a flight board in a free tool would have sounded rid
- **Same arc, for chips:**
  - 2018 — DARPA IDEA funds OpenROAD at UCSD — the open backend TT actually runs on
  - POSH (parallel DARPA program) funded the open-source IP libraries
  - 2020 — Google + SkyWater open-source sky130 ; OpenLane wraps Yosys + OpenROAD + Magic

  - 2022 — Matt Venn launches Tiny Tapeout
  - 2026 — you, tonight, in this room
- sky130 is the CERN moment. Tiny Tapeout is the Arduino moment.
- We are roughly where KiCad was in 2013.
- You do not want to be the last person who couldn't design a chip without a license.

Slide 6 But the Commercial Stack Still Exists

Honest about the other half.

- Every production fab job runs on commercial EDA
- **Cadence:** Virtuoso (layout), Spectre (analog sim), Genus (synth), Innovus (P&R), Xcelium (digital sim), Tempus (signoff STA)
- **Synopsys:** Synplify (FPGA synth — what you use at NU), Design Compiler (ASIC synth), ICC2 (P&R), VCS (sim), PrimeTime (signoff STA), Custom Compiler (layout)
- NU has licenses for all of it. These tools are excellent.
- **What they cost** (per seat / year, ranges — pricing is NDA):
  - Standard Cadence bundle ∼$15K (entry tier)
  - Generic high-end EDA seat ∼$100K (industry average)
  - Mixed-signal Virtuoso seat $200K+
  - Cadence SoC Encounter GXL flagship ~$750K (2007 figure, still cited)
- A single high-end seat runs from a hundred thousand to several hundred thousand dollars per year.
- A mid-sized team running a real ASIC tapeout pays one to five million dollars a year in EDA licenses alone
- Largest deals (Synopsys × Intel) exceed one billion dollars total.
- **Hedge it verbally:** "Vendors don't publish list prices — these are forum and court-exhibit numbers, but t
- Sources: edaboard.com EDA pricing threads; Quora chip-design Q&A; Wing VC industry brief on Synopsys + Cadence.
- **The catch for students using NU's academic licenses:**
  - No commercial use. Full stop. If you spin out a startup, every cell has to be re-built under a paid license.

  - PDKs (TSMC, GF, Samsung) are under NDA — you cannot open-source the layout, share .lef/.lib/.gds, or take files off the firewalled university network
  - Vendors can require sign-off before you publish a paper that shows tool screenshots or methodology
  - NU IP policy: anything you build with university resources belongs to NU. Tech transfer decides if you get it back.
  - Even if the university gives you the IP back, the design depends on PDK files you cannot legally take w

- This is why open tools + sky130 matter. You walk out of this room owning everything — the Verilog, the layout, the GDS, the right to publish or commercialize, all of it.
- Not because Cadence/Synopsys are bad. Because the open stack is the only one where the keys travel with you.

Slide 7 And You Can Still Take It Anywhere

Important nuance — because someone is going to ask.

- Tiny Tapeout is not a walled garden. It's a free public driveway.
- The toolchain (Yosys, OpenLane, OpenROAD, Magic, Verilator) is Apache 2.0 / MIT / BSD
- Yosys does not care if your repo is public — you can run the same toolchain on a private commercial MPW
- **The four real commercial paths from a TT prototype:**
  - Efabless MPW shuttles (sky130, GF180MCU)
  - Muse Semiconductor TSMC shuttles (real TSMC nodes, takes open-tool GDS)
  - GlobalFoundries 22FDX university and startup shuttles
  - Direct to TSMC / Samsung / GF with Cadence-Synopsys, the RTL transfers
- **The catch:** layout is not portable. Same way an Arduino sketch's PCB doesn't translate to a custom 4-layer board — you re-harden against the new PDK
- **IP nuances — be honest:**
  - Apache means anyone, including a competitor, can fork your TT design and ship it
  - Want closed-source silicon? You cannot *start* on TT, but you can use the *same open tools* on a private shuttle
  - File patents before you push to a TT repo — the push is public disclosure, the patent clock starts
- The university IP problem almost evaporates. The design is already public, already royalty-free, tech transfer has nothing to claim.

- Compare to Cadence on a TSMC PDK where every file is encumbered, every screenshot is NDA'd, every spinout has to re-implement from scratch.
- TT defangs the IP problem by making the IP irrelevant.

Slide 8 Today's Plan

Here's the plan for the next four hours. We start with how we got here — vacuum tubes, silicon, open PDKs — so the rest makes sense in context. Then a brief talk about transistors: what they are, how they switch.

Then we start building. You will build a transistor in SiliWiz — the same way the fab does it, layer by layer. Then we combine transistors into logic gates: NAND, flip-flops, a full adder. We simulate them in Wokwi.

Then the headline: in the next 90 minutes from when we sit down at our laptops, every single person in this room will have made a real integrated circuit in 130-nanometer CMOS. No matter how small the functionality, that is a real chip. That's the point.

After that we look at where HDLs come in — Verilog automates the placement of your transistor "Legos." And we finish by showing you how to run the whole pipeline locally on your own machine, and still send your GDS to Tiny Tapeout.

Slide 9 What Is Digital Communication?

Before we start building, let me give you 60 seconds of context. Everything in your laptop — every photo, every message, every calculation — is represented as sequences of ones and zeros. That idea is not new.

In the 1840s, the telegraph encoded language as dots and dashes — two symbols, transmitted electrically over a wire. As we got faster, the encoding got more sophisticated: Baudot code, ASCII, and eventually raw binary. But the principle never changed. Two states. On and off.

Slide 10 From Logic to Computation

In the 1930s, Turing showed that any computation can be performed by a machine operating on binary symbols. Boolean logic — AND, OR, NOT — became the mathematical foundation.

By the 1940s, engineers built these machines for real using vacuum tubes. ENIAC used over 17,000 of them and filled an entire room. A vacuum tube is just a big, hot, expensive switch. It works, but it does not scale.

Slide 11 The Transistor Revolution

The transistor replaced the vacuum tube in 1947. Same function — a switch — but solid-state. Smaller, faster, cheaper, more reliable. By the 1960s you could put multiple transistors on a single chip. By the 2020s, tens of billions. That progression is called VLSI — Very Large Scale Integration.

But as the chips got more complex, the manufacturing process became closely guarded. The recipe for building transistors at a given foundry is called a PDK — a Process Design Kit. And historically, PDKs were locked behind NDAs and six-figure license fees. Hold that thought.

Slide 12 How a Transistor Works

- A MOSFET is a voltage-controlled switch.
- Three terminals: gate , source , drain
- Put a voltage on the gate, it induces a field, the field lets current flow from source to drain.
- It is inherently a switch.
- Gate low → no channel → **OFF**
- Gate high → channel forms → **ON**
- All the specifics — threshold voltage, channel physics, oxide thickness — live in CE 391. Take it.
- Today we only need two facts: current flows from source to drain, and that flow is either on or off dependin
- We need that to talk about how these are made. (next slide)
- Two flavors: NMOS on with high gate, PMOS on with low gate. Pair them → CMOS , the building block of every digital chip you'll touch tonight.
- Transistor equals switch. Switches equal logic. Logic equals everything else.

Slide 13 How Is a Transistor Actually Made?

Quick tour of how a transistor is actually built. The fab grows and patterns layers on a silicon wafer using photolithography. Oxidize the surface — grow a thin SiO_2 layer. Coat it with photoresist. Shine UV through a mask. Develop the resist to reveal a pattern. Etch or implant. Then repeat with the next layer: diffusion, polysilicon, metal.

Each layer is aligned to sub-nanometer precision. A modern chip may take 80 or more mask steps. The sky130 process we use tonight has about 40. You don't need to understand any of this physics tonight — I just want you to have a mental picture for what those layers are when you draw them in SiliWiz in a few minutes.

Slide 14 Why Does This Matter for Us?

Why am I telling you all this? Because in 30 minutes you are going to draw a transistor in SiliWiz, and I want you to know the layers you're drawing are real. Each layer has design rules — minimum widths, minimum spacings, minimum overlaps — and those rules live in something called a PDK, a Process Design Kit. The PDK is the foundry's spec.

You do not need to run a fab tonight. You need to know what the fab expects, and the tools enforce it for you. That's the whole game.

Slide 15 The Secret Sauce: PDKs

The PDK is the foundry's complete spec — transistor models, design rules, layer stackups, parasitics, the whole recipe. Historically PDKs were locked behind NDAs and six-figure license fees. Only large companies and well-funded VLSI labs had access. No PDK equals no chip. Period.

Then in 2020, Google and SkyWater open-sourced the sky130 PDK — a full 130-nanometer process. Free. No NDA. No license. Anyone can use it. That is what made everything tonight possible. We are going to design a chip on an open process, with open tools, and submit it for real fabrication. No secret sauce.

Slide 16 From Open PDK to Open Fabrication

OK, so we have an open PDK. Great. But fabrication is still expensive. A traditional shuttle run at 130 nanometers costs 30 to 50 thousand dollars for a tiny area. A full mask set can be half a million. Only big companies could afford to tape out.

Efabless changed this. They organized shared shuttle runs — hundreds of designs on one wafer, splitting the cost. Google sponsored free shuttles through Efabless starting in 2020. Thousands of open-source designs got fabricated for the first time. But even a free shuttle required months of setup and serious EDA expertise. The barrier dropped, but it was still high.

Slide 17 Tiny Tapeout: Chip Design for Everyone

Then Matt Venn created Tiny Tapeout. Same idea — shared wafer — but radically simplified. Your design fits in a tile, about 160 by 100 microns, roughly a thousand gates. Hundreds of tiles share one chip. A tile plus a dev board plus your fabricated chip shipped to you costs about 150 dollars.

Let me say that again. Traditional fab: half a million dollars. Efabless shuttle: free but hard. Tiny Tapeout: 150 dollars, and you can do it tonight with a browser and a GitHub account.

Slide 18 What You Need

Last check before we start. You need a laptop plugged in, a GitHub account (create one at github.com if you don't have one), and Chrome or Firefox. That's it. No installs. No setup. For Levels 1 through 3 everything runs in the browser. Level 5 is the only one where you install something locally, and that's a stretch goal.

Slide 19 Level 1: Draw a Transistor in SiliWiz

Alright, let's build something. Open your browser and go to SiliWiz — the link is in the handout. You're about to draw a MOSFET. Follow along on the projector: select n-diffusion, draw a horizontal rectangle (the channel). Switch to polysilicon, draw a vertical rectangle crossing it (the gate). Switch to metal, place three contacts: source on the left, drain on the right, gate on top of the poly.

Then switch to Cross-section view — see the layer stack? That's what the fab actually builds. Then switch to Simulation view and sweep the gate voltage. Watch the threshold — that's where the transistor turns on.

Think of it this way: before you plug a resistor into a breadboard, you need to know what a resistor is. SiliWiz is the same idea. And this is not a toy — you can download the SPICE netlist or even an STL of the layers. The layers you draw are the same ones in the sky130 PDK. **Give the room 25 minutes here.**

Slide 20 Level 1: What You Just Built

Quick reset: the rectangle you just drew is a real NMOS transistor, laid out the same way the fab would build it. The n-diffusion is the source and drain — doped silicon. The polysilicon crossing it is the gate. The metal contacts are how the outside world reaches each terminal.

When you swept the gate voltage in Simulation view, you watched the channel form. Below threshold: no current. Above threshold: current flows from source to drain. That is the switch. Every transistor in every chip on this planet works exactly this way. You just built one.

Slide 21 How a NAND Gate Is Built on Silicon

This slide is optional context for the curious. A CMOS NAND is just four transistors in a specific topology: two NMOS in series (the pull-down) plus two PMOS in parallel (the pull-up). On silicon: two n-diffusion regions sharing a contact (series), two p-diffusion regions side by side (parallel), polysilicon gates crossing both, and metal wires connecting them.

NOR is the dual: two NMOS in parallel, two PMOS in series. Every standard cell in a PDK is one of these arrangements, hand-optimized. Full cell layout is a CE 391 topic — we are going to build them in Wokwi instead.

Slide 22 Level 2: NAND Gate — Truth Table

Open the NAND gate Wokwi project from the handout. Toggle inputs 1 and 2 with the DIP switches. Find the combination that turns the red LED on. Verify the truth table: NAND outputs HIGH for every combination except when both inputs are HIGH. That is two transistors. That is the foundation of everything. Give 3–5 minutes here.

Slide 23 Level 2: The Flip-Flop — Adding Memory

Now open the flip-flop project. It looks more complex but the idea is simple: it *remembers* a value. When Enable is HIGH, the output follows the data input. When Enable goes LOW, the output holds its last value. That is memory. One bit of memory.

The NAND gate you just saw is *combinational* — no memory, output depends only on current inputs. The flip-flop is *sequential* — it has state. Combinational plus sequential: together they can compute anything. That is a Turing machine.

Slide 24 Level 2: The Full Adder — How Fast It Scales

Last Wokwi demo before you build your own. Open the full adder. Toggle A, B, and Carry-in. Watch the Sum and Carry-out LEDs. Count the gates: two XOR, two AND, one OR — five gates to add two bits. Chain 32 of these and you have a 32-bit adder. Chain 64 and you have the ALU in your laptop.

Now look at the wiring on screen. Imagine drag-and-dropping 64 of these. You can see why this does not scale. That is why we need Verilog — describe the logic in text, let the compiler wire the gates. But first, you're going to build one by hand.

Slide 25 Level 2: Build Your Own Gate in Wokwi

Now it's your turn. Open the blank Wokwi template from the handout. **Save immediately — Ctrl+S, right now**. If you don't save, you don't get a project ID, and you cannot submit. I'll wait. *(Pause. Wait for the room. Confirm with TAs.)*

Good. Now build a NAND from scratch: press the + button, search for 74HC00 (a quad NAND chip), place it. Wire it up: IN0 to pin 1A, IN1 to pin 1B, pin 1Y to OUT0. Don't forget power: VCC to pin 14, GND to pin 7. Click Play, toggle inputs, verify the NAND truth table. Give 15 minutes here.

Slide 26 Level 3: Fork the Template

Level 3 starts now. This is the moment your logic gate becomes a real chip layout. You're about to push your Wokwi design to GitHub, and a CI pipeline will run it through the exact same tools that professional chip designers use — Yosys for synthesis, OpenLane for place-and-route, DRC for design rule checking. At the end you get a GDS file. That's the file format the fab reads to make your chip.

First: go to github.com/TinyTapeout/ttsky-wokwi-template. Click Use this template, then Create a new repository. Name it whatever you want. Make sure it's set to Public.

Slide 27 Level 3: Link Your Wokwi Design

Open info.yaml in your new repo. Find the wokwi_id field under the project section. Go back to your Wokwi project — the number in the URL bar after /projects/ is your ID. Paste it in. Fill in title, author, and description. Commit.

TAs: the single most common failure point in the entire workshop is the wrong wokwi_id. It must be the number from the URL, not the project name. If a GHA build fails, check this first.

Slide 28 Level 3: Watch the Action

Go to the Actions tab in your repo. You should see a workflow called gds running — yellow dot. Takes about 5 minutes. While we wait, here's what it's doing: first it fetches your Wokwi schematic and converts it to Verilog. Then Yosys synthesizes it — maps your logic to actual transistor-level cells from the sky130 standard cell library. Then OpenLane does place-and-route — arranges those cells on the chip and connects them with metal wires. Then DRC checks that everything meets the foundry's manufacturing rules. And finally it generates a 3D viewer.

When it turns green, you're good. If red, check the logs — most common: wrong wokwi_id, design too large, or a floating port in your Wokwi schematic.

Slide 29 Level 3: View in 3D

Once both gds and docs show green checkmarks, go to Settings, then Pages, and set the source to GitHub Actions. Then visit your GitHub Pages URL.

You're looking at a 3D render of your actual chip layout. Toggle the layers on and off. See if you can find an inverter cell — two MOSFETs, drain to drain, exactly what you drew in SiliWiz 45 minutes ago. Everything connects.

Slide 30 From Truth Table to Gates — Automatically

One last Wokwi trick before we leave it behind. Remember the NAND truth table you wired by hand? What if you had a more complex truth table — say, four inputs and seven outputs for a seven-segment display decoder?

There's a tool called the Wokwi LUT Generator. You write your truth table in JSON, and it automatically generates the Wokwi schematic — all the AND gates, OR gates, wiring, everything. It uses boolean algebra (Quine-McCluskey) to minimize the logic, then builds the circuit for you.

This is exactly what Verilog and Yosys do. You describe what the logic should do. The tool figures out how to build it. That is the jump we are about to make.

Slide 31 OK — Let's Talk Verilog

Alright. Everything we have done so far has been drag-and-drop. Wokwi is a great starting point — you built a NAND gate, you generated a GDS, you saw your chip in 3D. That is real. That works. But let's be honest — you are not going to drag and drop your way to a CPU.

Wokwi caps out at about 20 gates. Real designs have thousands. Counters, state machines, VGA controllers, processors — all of that needs a hardware description language. That language is Verilog. Same idea as going from the Arduino block editor to writing real C++. You describe what the hardware should do in text, and a compiler called Yosys turns it into gates.

Levels 4 and 5 are stretch goals. If we get through them tonight, great. If not, they are in the handout and you have until May 9.

Slide 32 Level 4: What Is This Code?

This is the densest slide in the workshop. Go slow. Open vga-playground.com on the projector and pick Stripes. Walk down the table line by line.

The module declaration at the top — that's the top-level box. Its inputs and outputs are the 26 TT tile pins. The pix_x and pix_y wires are the current pixel position — the VGA sync generator sweeps 640 columns and 480 rows, 60 times per second. The counter is a 10-bit register that goes up by one per frame — this is what makes things move. A wire is a connection; a reg is memory.

The key line is moving_x equals pix_x plus counter. Adding the counter every frame shifts the pattern right by one pixel. Your eye sees stripes scrolling left. Then the assign R picks bit 5 of the

shifted position — bit 5 flips every 32 pixels, so you get a 32-pixel stripe. The square bracket is the stripe width.

At the bottom, always at posedge vsync runs once per frame and bumps the counter. That's the heartbeat. Reassure people: *you do not need to memorize this. The goal is just to know enough to change one line.*

Slide 33 Level 4: Two Kinds of Verilog

Two things to know about Verilog before we move on. Everything is one of two flavors. Combinational uses `assign` or `wire` — evaluated every time inputs change. Think light switch: flip input, output changes. Just wires and gates in hardware.

Sequential uses `always @(posedge clk)` — evaluated once per clock edge. This is how you build flip-flops and counters — things that remember. The counter we just touched is sequential.

And the trap: some things simulate perfectly but never synthesize. `#10` delays — there's no such thing in real gates. Division — use bit shifts. Large memories — a full framebuffer is 38 KB, way too big for one tile.

Slide 34 Level 4 Goal: Reverse the Stripes

Find the line: `wire [9:0] moving_x = pix_x + counter`. Change the plus to a minus. That's it. One character. Look at your screen — the stripes reversed direction. You just modified hardware by editing one character of text.

Now play with it. Change moving_x[5] to [4] for 16-pixel stripes, [6] for 64-pixel. Change which colors are on. Break it and fix it — coloured underlines mean syntax errors and the screen freezes on the last valid state. Smart kids: go nuts. The handout has the full breakdown and more experiments.

Slide 35 Templates and Module Naming

Quick logistics slide. There are four templates depending on what you're building — Wokwi, Verilog on sky130, Verilog on IHP, and analog. For Level 4 you want the IHP Verilog template since VGA designs use that shuttle.

Module naming is the most common gotcha after wokwi_id: your top module must be named `tt_um_YOURGITHUBNAME_projectname`, and the `top_module` field in info.yaml must match *exactly*. Mismatch = red CI.

The four files you actually edit: project.v (your design), info.yaml (metadata), docs/info.md (how it works), test/test.py (at minimum, add cocotb.pass_test).

Slide 36 Level 4 Goal: GDS on GitHub Actions

Walk through on screen. Fork the IHP Verilog template. Pages source: GitHub Actions. Copy the Verilog into project.v, rename the module. Update info.yaml and docs/info.md. Add cocotb.pass_test in test/test.py. **Important:** for VGA designs, also add hvsync_generator.v to PROJECT_SOURCES in test/Makefile or synthesis fails on undefined module.

Commit and wait for green. Same flow as Level 3, but now you wrote the code instead of dragging gates. Green CI means your Verilog synthesized, passed DRC, and is fab-ready.

Slide 37 Level 4: Submit

If your GDS action is green, you can submit right now. Go to app.tinytapeout.com/projects/create, log in with GitHub, paste your repo URL, click Submit. If you have a coupon, enter it. You can update any time before May 9 by committing again and clicking Submit a Revision.

Slide 38 How Submission Actually Works

Now we walk through exactly how you go from green checkmark to fabricated chip, because I don't want anyone guessing.

Step one: GitHub Actions are green — both gds and docs. That means your design compiled, met timing, and passed DRC. Fab-ready. Step two: tokens. On May 5 we distribute submission tokens. Each token reserves one tile. If you don't have a token, you can pay directly — about $150. Step three: go to app.tinytapeout.com/projects/create, log in with GitHub, paste your repo URL, enter your token or payment. Step four: iterate any time before May 9. Just commit and click Submit a Revision. After May 9 the shuttle closes. Your tile gets fabricated on a shared wafer. Six to nine months later, board in the mail with your chip on it.

Slide 39 Your Tile's Interface: 26 Pins

Every TT tile has the same fixed 26-pin interface. Your design plugs into it. clk and rst_n are obvious. ena is high when your tile is selected on the chip. ui_in is your 8 input pins, uo_out is your 8 output pins. Then there's an 8-bit bidirectional bus — uio_in, uio_out, and uio_oe (output enable: 1 means drive, 0 means listen).

Three rules. Unused outputs must be assigned 8'b0 — floating outputs fail DRC. If you're not using bidir, set uio_out and uio_oe both to 8'b0. And use rst_n (active-low) — no initial blocks. There's no power-on reset on real silicon.

Slide 40 A Note on IP and Open Source

Quick legal note. TT requires Apache 2.0. Your design and documentation will be publicly published. You keep ownership, but anyone can use, modify, and build on it. Publication counts as public disclosure for patent law — not compatible with NDAs.

For students: almost always fine. You're building a learning project. For research labs: **talk to your tech transfer office before submitting.** If you're working on something that might be patented, do not push it to a TT repo first — you'll burn the patent novelty window.

Slide 41 Testing Your Design

One more thing before we wrap. Testing. Before you submit, you should verify your design actually does what you think. TT uses cocotb — a Python framework that drives your Verilog inputs and checks outputs. You write a test that sets inputs, waits a clock cycle, asserts outputs.

At minimum: just add `cocotb.pass_test()` so CI passes. For real verification, write asserts for every input combination. The handout has the full guide.

Plug for Tapeout Day: optional group hackathon before May 9. Bring your laptop. We write tests, fix bugs, polish designs, submit together. Nobody gets left behind.

Slide 42 Tapeout Assignments — Let's Talk

Pause the slides here. Time for the conversation. We have a limited number of tiles for this workshop and this round is digital only. I need to know who's committing tonight so I can hand out tokens correctly on May 5.

Go around the room: who is committing to a tapeout this cycle? What's the design idea, in one sentence? Solo or pairing up? Who needs help picking a project? It's totally fine to still be deciding — but we need to know by the end of today. Analog or mixed-signal: come to the advanced session, different flow.

Slide 43 Level 5: Install OSS CAD Suite

Everything we've done so far has been in the cloud — GitHub Actions ran the tools for you. But what if you want to see what's actually happening, or catch errors before you push? OSS CAD Suite is a local download with the exact same Yosys and iverilog that TT's CI runs.

Pick the right file for your platform: linux-x64 for most, darwin-arm64 for Apple Silicon Macs. Untar, source the environment file, run `yosys -version`. **Mac heads up:** the binary may be quarantined — if so, allow it in System Settings → Privacy & Security.

Slide 44 Level 5: What Is OSS CAD Suite?

If you've used Quartus or Vivado, this will feel different. There is no GUI. No project wizard. No schematic editor. OSS CAD Suite is a bag of command-line tools — yosys for synthesis, iverilog for simulation, gtkwave for waveforms, nextpnr for FPGA place-and-route.

Think gcc plus gdb plus make, not Visual Studio. Write Verilog in a text editor — VS Code with the Lushay Code extension is the closest you'll get to an IDE — compile from the terminal, debug by looking at waveforms.

Verilog is native. SystemVerilog and VHDL work via plugins, but for TT plain Verilog is all you need. The key advantage: it's the exact same toolchain that TT's GitHub Actions runs. Synthesize locally and you're running the same Yosys that decides whether your design passes CI.

Slide 45 Level 5: FPGA → ASIC — What to Fix

If you've written Verilog for an FPGA before, here are the gotchas when porting to ASIC. Flops on an FPGA come up at zero — you can rely on it. On ASIC there is no power-on state, so you must use an explicit reset. TT uses active-low reset, so the signal is rst_n. `initial` blocks work in simulation and on FPGA but are silently ignored on ASIC — remove them. Division synthesizes on FPGA but not on the open ASIC flow — use bit shifts. `#delay` is simulation-only.

Slide 46 Level 5: Simulate and Synthesize

The workshop repo has examples in the examples/ directory — a 4-bit counter and a testbench. Activate the tools first: source oss-cad-suite/environment. You need this in every new terminal. **Important:** cd into the workshop repo before running commands — oss-cad-suite also has an examples/ directory that is unrelated.

To simulate: iverilog -o sim examples/tb.v examples/project.v, then vvp sim. You should see three lines — count equals 4, pause holds it, reset brings it back to 0. Then gtkwave tb.vcd to see the waveforms. **When gtkwave opens you'll see a blank screen** — expand tb, then dut, click count, clk, rst_n, uo_out, and Append for each. Then Shift+Alt+F to zoom to fit.

To synthesize: yosys -p "read_verilog examples/project.v; synth -top tt_um_yourname_counter; stat; show". The stat command prints the cell inventory. The show command pops up a gate-level schematic. Same thing GitHub Actions does, but you can see it.

Slide 47 Level 5: Get Into Tiny Tapeout

Same submission flow as Level 3 and Level 4, but now with code you wrote yourself locally. Fork the appropriate Verilog template, copy your .v file in, update info.yaml so top_module matches your module name, add cocotb.pass_test, commit, wait for green, submit on app.tinytapeout.com.

Slide 48 Key Deadlines

Closing slide. The dates that matter: May 1 is the IEEE form — name and one-line description. May 5 is token distribution. May 9 is the hard deadline — after this the shuttle closes and your design goes to the fab. After May 9 you cannot iterate.

Handout has homework and stretch goals in the appendices — more Wokwi designs, aspirational VGA projects, even an 8-bit CPU that fits on one tile. Take a look this week. Thank you all for coming. TAs will stick around for questions.

Advanced Session

Friday April 10, 2026

Slide 1 Tiny Tapeout: Advanced (Analog Design)

Welcome back. Tonight we go deep. In the beginner session you went from a transistor to a synthesized GDS file using browser tools. Tonight we pull back the curtain and do what the tools were doing for you — by hand.

By the end of tonight, you will have built a CMOS inverter from scratch: schematic in XSCHEM, simulation in NGSPICE, physical layout in Magic, parasitic extraction, and submission to Tiny Tapeout. This is the same flow that produced every standard cell in the sky130 library.

But first — is everyone's Docker running? If you haven't installed Docker yet, start now. TAs, please help anyone who's stuck. The image is 12 GB so this should already be downloading.

Slide 2 About Me

Same pitch as the beginner session, abbreviated if most of you were there: I work on ultra-low-power systems in the VAK lab with Professor Arora. The reason I care about chip design is that off-the-shelf silicon can't get low enough for what we need — COTS caps out around a Cortex-M0, and that's still too much for a battery-free sensor. Tiny Tapeout is what lets a systems person actually do something about that.

Slide 3 What You Can Learn at Northwestern

Same framing I gave the beginner room. Northwestern has the courses to teach you almost every step of chip design — everything except actually taping out. For that you have to join a VLSI lab. But what if you're not a VLSI person? The skills, the licenses, the months sunk into one design — chip design has been a destination, not a tool. Tonight we change that. Show of hands: who has never touched analog before? Digital?

Slide 4 What This Lab Is / Is Not

Let me be upfront about what this is. I am not here to teach you electrical and computer engineering in a few hours. By your junior or senior year, you have already learned the concepts — transistors, logic, circuit analysis. What I'm here to do is show you the *tools* that let you do this stuff in open source.

For those of you who don't have that background yet: you can absolutely keep up. Everything is guided. You'll learn by doing.

For those of you who do: this is where it gets interesting. These open-source tools are obviously not up to par with the billion-dollar commercial EDA industry — yet. That means there's room to contribute. If you find something broken, that's not a bug report, that's a research opportunity.

This session is a full-stack analog design walkthrough. Schematic to silicon. But it's not a VLSI course, and nobody expects your first layout to be perfect. We'll iterate.

Slide 5 Open Source Is Coming for Chip Design

Two parallel timelines, one already happened. KiCad started in 2004 as a French university toy — nobody serious touched it. Then in 2013 CERN adopted it, hired engineers, contributed code. By 2018 Adafruit and Arduino were publishing all their boards in KiCad. Today SpaceX, Google, Meta all use it. Twenty years ago telling a PCB engineer they would design a flight board in a free tool would have sounded ridiculous — it is now routine.

The same arc is happening for chips, just earlier. In 2018 DARPA's IDEA program funded OpenROAD at UCSD — the open backend Tiny Tapeout actually runs on. In 2020 Google and SkyWater open-sourced sky130. OpenLane wraps Yosys plus OpenROAD plus Magic into one flow. In 2022 Matt Venn launches Tiny Tapeout. And tonight, you. We are roughly where KiCad was in 2013. sky130 is the CERN moment. Tiny Tapeout is the Arduino moment.

The point: you don't want to be the last person who couldn't design a chip without a license. **And in the analog flow we're about to do, you are even earlier on the curve — this is the part open tools are still rough at, which means there's room to contribute.**

Slide 6 But the Commercial Stack Still Exists

Honest about the other half. Every production fab job runs on commercial EDA. Cadence Virtuoso, Spectre, Genus, Innovus on one side. Synopsys Design Compiler, ICC2, PrimeTime, VCS on the other. Northwestern has licenses for all of it. These tools are excellent — faster, better timing closure, what you'll touch the day you take a job at a fab.

But we're not using them tonight. Not because they're bad — because the open stack is the only one where *you* keep the keys when you walk out of this room. The skills transfer directly. The day you have a reason, you can re-harden in Cadence. The XSCHEM schematic you're about to draw has a one-to-one analog in Virtuoso.

Slide 7 The Session Arc

Here's the arc for tonight. We start with a quick digital warm-up — VGA Playground and OSS CAD Suite, the same tools from the beginner session. Then we do something most synthesis tutorials skip: the mixed-signal reveal. I'll show you exactly what synthesis CAN'T build, and why that matters.

Then we switch to the analog flow. XSCHEM for schematics. NGSPICE for simulation. Magic for physical layout. Parasitic extraction. And finally, GDS submission.

The key insight is this: the inverter you drew in SiliWiz and the inv_1 cell that Yosys inserts during synthesis are the same circuit. One was built by hand. The other was built by a person years ago and put in a library. Tonight, you are the person building it.

Slide 8 TT Tile Interface: The HDL Contract

Quick reference slide — the HDL contract. Every Verilog tile must implement this exact module interface. Same 26 pins from the beginner session: clk, rst_n, ena, ui_in[7:0], uo_out[7:0], and the

bidir bus uio_in/out/oe. Module name must be tt_um_YOURGITHUBNAME_project. Unused outputs must be assigned 8'b0 or synthesis errors out. Two templates: ttsky-verilog-template for sky130, ttihp-verilog-template for IHP.

This is reference — I don't dwell on it. It's here so people can flip back during the practical part.

Slide 9 Part 1: VGA Playground — Verilog in Practice

Quick warm-up. If you didn't finish Level 4 in the beginner session, open vga-playground.com now and get a design working. If you did, improve it — change colors, change speed, try a different preset. You have 10 minutes. This is intentionally low-pressure while Docker images finish downloading on the laptops that are still pulling.

Slide 10 Part 2: Install OSS CAD Suite

Now let's look under the hood. OSS CAD Suite gives you the exact same Yosys that GitHub Actions runs. Download the right file for your platform — linux-x64 for most of you, darwin-arm64 for Mac. Untar it, source the environment file, and run `yosys -version` to verify. Mac people: if the binary is quarantined, allow it in System Settings.

Slide 11 Part 2: OSS CAD Suite vs. Quartus / Synplify

If you've used Quartus or Vivado, this will feel different. There's no GUI. No project wizard. No schematic editor. It's a bag of command-line tools — yosys for synthesis, iverilog for simulation, gtkwave for waveforms.

Think gcc plus gdb plus make, not Visual Studio. You write Verilog in a text editor, compile from the terminal, and debug by looking at waveforms. VS Code with the Lushay Code extension is the closest you'll get to an IDE.

Slide 12 Part 2: The Local Workflow

Here's the workflow. **Important:** make sure you're in the workshop repo directory, not oss-cad-suite — both have an examples/ folder but they're different. Simulate with iverilog, view waveforms in gtkwave — remember, expand tb then dut, append your signals, and Shift+Alt+F to zoom to fit, or you'll see a blank screen.

Synthesize with yosys. The stat command prints your cell inventory. The show command pops up a gate-level schematic so you can see how the gates are wired.

What you will NOT get locally: timing analysis. TT's CI runs OpenSTA for static timing analysis after place-and-route. Locally, you get logic correctness. Timing sign-off happens in the cloud.

How do you know if something is synthesizable? If yosys errors, it can't build it. Delays, division, initial blocks, large memories — those will fail or be silently dropped. How do you know if CI will pass? Run the same yosys command locally. If it works here, it works there.

Slide 13 Part 2: What `stat` Tells You

After you synthesize, Yosys prints a cell inventory. Look at the cell names — they all start with sky130_fd_sc_hd. That's the sky130 standard cell library. Yosys doesn't invent new transistor arrangements. It picks from a fixed menu of pre-built cells and wires them together.

See that inv_1? That's an inverter. Someone built that cell by hand — drew the schematic, simulated it, laid it out, ran DRC, ran LVS, extracted parasitics, and characterized it. Yosys just picks it off the shelf. We are about to do exactly that process ourselves.

Slide 14 Part 2: From Tiny Tapeout to Production

One more thing before we move on. Some of you are thinking: OK, Tiny Tapeout is cool for learning, but what if I actually want to take this to production? What if I'm a startup and I want to go to TI or TSMC?

The answer: the workflow is the same. RTL, synthesis, place-and-route, DRC, LVS, tapeout. That flow doesn't change. What changes is the tools — Synopsys Design Compiler instead of Yosys, Cadence Innovus instead of OpenLane, PrimeTime for signoff timing. And the PDK — you'd be working with TSMC or GlobalFoundries under NDA, not sky130.

But here's the thing: your Verilog is portable. Your testbenches are portable. Your design skills are portable. The layout is not portable — you re-harden with the target PDK. But the RTL that generated it will work anywhere. That's why learning this flow matters even if you never ship on sky130.

Slide 15 Tapeout Requirements — Let's Talk

Pause the slides here for the tapeout-requirements conversation. We have a limited number of tiles for this round. Before we go further into the analog flow, let's make the plan concrete.

Go around: who is committing this cycle? Digital, analog, or mixed-signal? Analog uses the tt10-analog-template and a different submission flow with a manual review step. One-tile or multi-tile? Analog macros often need 2x1 or larger. One-sentence design idea? Solo or pairing up?

Tokens go out May 5. Submission deadline May 9. We need a headcount today. **Reminder for research-lab folks:** analog designs are publicly published under Apache 2.0 just like digital. If you're touching anything that might be patented, talk to your PI and tech transfer office *before* you push it to a TT repo.

Slide 16 Part 3: The Mixed-Signal Reveal

Here's the thing that synthesis tutorials don't say out loud. Synthesis can ONLY produce circuits that switch between two stable voltage levels. Full stop. If your circuit does anything else — oscillates, amplifies, converts between analog and digital — it cannot be synthesized.

Ring oscillators — the frequency depends on transistor physics. PLLs — feedback across analog and digital. ADCs and DACs — they convert between continuous and discrete, which is inherently analog. ESD protection — diodes and clamps. I/O buffers — they have to drive real off-chip loads.

Now think about any real chip. Your phone. A microcontroller. A sensor IC. Every single one has at least a power-on reset circuit, I/O buffers, and usually a clock source. All analog. Every chip that contains any of those is a mixed-signal ASIC. That's almost every chip ever manufactured.

Slide 17 Part 3: What This Means for sky130

Sky130 has two device libraries. `sky130_fd_sc_hd` is the standard cell library — digital. Yosys picks from here. Pre-built, never touched by hand.

`sky130_fd_pr` is the foundry primitives — analog. Raw transistors, resistors, capacitors. You place these by hand in XSCHEM and simulate them in NGSPICE.

When you drew the inverter in SiliWiz, you were in the fd_pr world. When Yosys inserts an inv_1 cell, it's using something that was built the same way. The advanced flow is the fd_pr flow. You are doing by hand what Yosys does automatically.

Slide 18 The File Ecosystem: Design Files

Reference slide. There are a lot of file extensions in the analog flow and people get lost. Walk down the table once so they recognize each name when it shows up later. The headline: `.v` is your Verilog source, `.sch`/`.sym` are XSCHEM, `.spice` is what NGSPICE eats, `.raw` is what NGSPICE produces, `.mag` is Magic layout, `.ext` is the intermediate Magic extraction.

Slide 19 The File Ecosystem: Output and Submission

Companion to the previous slide — the output side. `.gds` is the deliverable that goes to the fab. `.lef` describes your cell boundary and pins for the place-and-route step. `.def` is placement/routing data; TT provides templates. `.vcd` is a waveform log. info.yaml is project metadata that the CI pipeline reads.

Bottom of the slide is the flow in files: schematic to SPICE to raw waveforms (simulate); schematic to layout to extraction back to SPICE (extract parasitics); layout to GDS plus LEF (submit). That's the whole night in one diagram.

Slide 20 Timing, Models, and Characterization

Pure reference. These files all come from the PDK — you use them but never edit them. Liberty (.lib) files describe cell timing for synthesis. corner.sym selects the process corner — always set to `tt` (typical) for tonight. .magicrc loads PDK rules. .sdf is delay annotation for gate-level sim. .tcl files configure Netgen for LVS.

Don't dwell — people get glassy-eyed at this. The takeaway is: when something errors and mentions one of these, they know where to look.

Slide 21 Analog Tile Specifications

Important reference for anyone who's actually planning an analog tile. Minimum size is 1x2 (160 by 225 microns). You get up to 6 analog pins, and they must be used in order from `ua[0]`. Pin resistance has to stay under 500 ohms, capacitance under 5 pF, and you can drive at most 4 mA per pin.

Two supplies: VDPWR is the 1.8 V digital, VAPWR is the 3.3 V analog (optional, requires the 3v3 template). Metal 5 is reserved for the TT power grid — do not use it. Power rails are vertical met4 stripes from your bottom 10 microns to your top 10 microns, minimum 1.2 microns wide. Plan for about 0.1 V of PDN drop at 20 mA when you set your bias points.

Slide 22 Analog Submission Requirements

Logistics for analog tiles. You submit a GDS and a LEF (export from Magic with `-pinonly`). Three template variants depending on whether you need 3.3 V and how big your tile is.

Pricing: about 140 euros base for the minimum 2-tile area, 40 euros each for the first two analog pins, 100 euros each for additional pins. Use the calculator on the TT app to plan.

Two rules to repeat: unused digital outputs equal zero, no floating pins. And: **analog tiles get a manual review** on top of automated DRC, so submit early — not on May 9.

Slide 23 How to Purchase and Submit

Eight-step submission flow. Green CI on both gds and docs first. Then on May 5, get a token from your workshop organizer (each token reserves one tile). Go to app.tinytapeout.com/projects/create, log in with GitHub, paste your repo URL, enter token or pay. Iterate until May 9 by committing and clicking Submit a Revision. Confirmation email means you're on the shuttle. Six to nine months later, the dev kit arrives with your chip on it.

Plug Tapeout Day — the optional group hackathon I mentioned earlier. Bring your laptop, we ship together.

Slide 24 IP and Legal Considerations

Important for anyone working in a real research lab. Submitting to TT is a binding contract. Apache 2.0 means your design and docs are publicly published — anyone can use, modify, commercialize. You keep ownership of your individual tile, but the combined chip belongs to Tiny Tapeout Ltd.

Public disclosure starts the patent clock. In the US you have a one-year window. In many other countries, public disclosure kills patentability immediately. You also indemnify TT — you guarantee you own the IP and that it doesn't infringe anyone else's. You handle export-control compliance (matters for crypto and dual-use). Jurisdiction is Israeli law, Tel Aviv courts.

Don't read this whole slide aloud — flag the highlights and tell people to read the terms before they push the Submit button.

Slide 25 Is Tiny Tapeout Right for Your Lab?

Decision aid for the research-lab people. Good fit if you want open-source hardware publication, educational prototypes, public demos, systems-level experiments, or you've already disclosed and plan to open-source anyway. Risky if you care about patents, commercial spinouts, industry partnerships, NDAs, or you have sponsor restrictions on publication.

Three things to do before submitting from a research lab: talk to tech transfer, check grant and sponsor IP clauses, verify no embedded third-party or NDA'd IP in your design. If your institution has approved open-sourcing and there's no sponsor baggage, your legal risk is low.

Slide 26 Part 4: Setup iic-osic-tools

Alright, let's build an inverter. Open your terminal. Walk through this live. Confirm Docker is running. Run `xhost +local:docker` (or the colima fallback). `cd iic-osic-tools && ./start_x.sh`. Wait for the container GUI to open.

If anyone is stuck on Docker: pair them with a TA. *Do not* let the whole room wait — move forward and TAs will catch people up. The TA troubleshooting doc has every Docker failure mode.

Slide 27 Part 4: Create a Project

Inside the container, three commands: mkdir -p /foss/designs/my_inverter, cd into it, run xschem. XSCHEM should open. If it does, you're ready.

Slide 28 Part 4: Place Transistors

Walk through live on the projector. Press Insert. Navigate to /foss/pdks/sky130A/libs.tech/xschem/sky130_fd_pr/ Select nfet_01v8.sym — place it. That's your NMOS. Insert again, select pfet_01v8.sym, place it above the NMOS. That's your PMOS.

Slide 29 Part 4: Add Pins

Insert ipin.sym, place on the left, press Q, set lab=in. That's your input. Insert opin.sym, place on the right, Q, lab=out. That's your output. Insert iopin.sym twice — one at the top labeled vdd, one at the bottom labeled gnd. Power and ground.

Slide 30 Part 4: Wire the Inverter

Now wire it up. Press W to enter wire mode. Both gates go to in. Both drains go to out. PMOS source goes to vdd, NMOS source goes to gnd. PMOS bulk goes to vdd, NMOS bulk goes to gnd. A filled dot means the connection is made — if you see a crossing without a dot, the wires aren't connected.

Slide 31 Part 4: Set Transistor Parameters

Select the NMOS, press Q. Set W equals 1, L equals 0.15. That's the minimum gate length for sky130. Now select the PMOS, press Q. Set W equals 2, L equals 0.15.

Why 2 for the PMOS? Because hole mobility is about half of electron mobility. Doubling the PMOS width makes the rise and fall times symmetric. This is the most fundamental analog design knob there is. Save with Ctrl+Shift+S as my_inverter.sch.

Slide 32 Part 4: Create a Symbol

Symbol → Make symbol from schematic. XSCHEM generates a symbol file — the black-box view of your inverter. Save with Ctrl+S. You'll need this symbol for the testbench.

Slide 33 Part 4: Build the Testbench

File → New Schematic. We build a testbench — a separate schematic that instantiates your inverter and feeds it signals. Insert your my_inverter.sym from the project folder. Place two voltage sources — one for Vdd, one for the input signal. Place a capacitor for the load and a ground symbol.

Critical: insert the corner symbol from the PDK. Press Q, set corner=tt. **If you skip this, NGSPICE fails with "model not found"** — it's the most common stumble in this part.

Slide 34 Part 4: Configure Component Values

Set Vdd to 1.8 volts. Set the input source to a pulse: from 0 to 1.8 volts, 1 ns delay, 100 ps rise, 100 ps fall, 5 ns high, 10 ns period. Set the capacitor to 10 femtofarads — a small load.

Slide 35 Part 4: Add Simulation Commands

Place a code_shown.sym block. Press Q and type the SPICE commands: `.temp 27`, then `.control`, `tran 0.1n 30n`, `write tb_my_inverter.raw`, `.endc`. Place lab_wire.sym labels on the in, out, and vdd nets. Go to Simulation, Set netlist dir, and point it at your project folder. Save as tb_my_inverter.sch.

Slide 36 Part 4: Run Simulation

Click Netlist. Then Simulate. NGSPICE output appears in a terminal window. If it says "run" and finishes without errors, click Waves. GAW opens. Drag v(in) and v(out) into the panel. You should see clean inversion — input goes high, output goes low, and vice versa. That's your inverter working.

If it doesn't: flat line at zero usually means the Vdd source isn't connected. "Model not found" means missing corner.sym. No waveform file means the netlist dir wasn't set.

Slide 37 Part 4: Add Measurements

Place another code_shown.sym and add the .meas lines from the handout. These measure propagation delay (tphl, tplh) and rise/fall times. Re-run Netlist then Simulate. The numbers print in the console. **Write them down — you'll compare them to the post-layout results later.**

Timing: target reaching this point by 7:15 PM. If you're running late, skip the measurements and let people read the handout.

Slide 38 Part 5: Key sky130 Layers

Now we go from schematic to silicon. Magic is the layout tool — you draw the physical shapes that get manufactured, the same layers you saw in SiliWiz, but now at full precision with real design rules.

Six layers to know: nwell for the PMOS body, ndiff and pdiff for the active source/drain regions, poly for the gate, contacts to connect up to li1, and m1 for the power rails. That's it. Every transistor-level layout uses these six. Launch Magic from the terminal.

Slide 39 Part 5: Draw the Layout

This is the longest single step of the night. Walk through live. Draw an nwell rectangle in the upper half — that's where the PMOS lives. Draw pdiff inside the nwell. Draw ndiff in the lower half, outside the nwell. Draw a poly stripe crossing both diff regions, target width 0.15 microns. Add contacts in the diff regions. Route li1 wires: drains to out, sources to supply, gates to in. Add m1 power rails at the top and bottom, with pin labels. Save with `save my_inverter` in the Magic console.

TAs walk the room actively here. Common mistakes: poly not crossing diff (no transistor formed); contacts not inside diff (open circuit); wires crossing without intent (short).

Slide 40 Part 5: DRC

In the Magic console, type `drc check`, then `drc why`. It tells you every rule violation. Most common: poly too narrow (widen to 0.15 micron min), diff spacing too small, missing well taps. Fix them, run again. Iterate until it's clean. DRC clean means the foundry can actually manufacture this — it's the same check that runs on every production chip.

Slide 41 Part 5: LVS

LVS — Layout vs. Schematic. This checks that your layout is electrically identical to your schematic. Go back to XSCHEM, click Netlist to generate the SPICE file. Then run the netgen command from the handout.

You want to see "Circuits match uniquely." If it says "Netlists do not match," check pin labels, source/drain orientation, and missing contacts. This closes the loop — your hand-drawn layout is provably identical to the schematic you simulated.

Slide 42 Part 6: Parasitic Extraction

Your schematic simulation assumed ideal wires — zero resistance, zero capacitance. But real metal wires have both. Parasitic extraction pulls the real R and C values from your layout and puts them into a new SPICE netlist. When you simulate with parasitics, you see the true performance of your circuit.

Run the extraction commands from the handout in Magic. It flattens the layout, extracts everything, and produces my_inverter_flat.spice.

Slide 43 Part 6: Run Post-Layout Simulation

Open your testbench in XSCHEM. Add a code_shown block with `.include my_inverter_flat.spice`. Disable the original symbol instance — select it, press Q, add `spice_ignore=true`. Then Netlist, Simulate, Waves. Compare to your earlier results.

Slide 44 Part 6: Compare Results

Your post-layout rise time, fall time, and propagation delay will all be slightly worse than the schematic simulation. That's expected — it's the accurate result. The schematic sim was optimistic because it assumed perfect wires. This is what the fab will actually produce.

If the degradation is much larger than expected, you have a parasitic hotspot — a long thin wire or a tight via. Widen it or add more contacts.

Slide 45 Part 7: Export from Magic

Last step. Export your layout. In the Magic console: `gds write my_inverter.gds` and `lef write my_inverter.lef`. Two files. That's the deliverable.

Slide 46 Part 7: Use the sky130 Analog Template

Fork the analog template on GitHub. Drop your GDS into gds/, your LEF into lef/. Update info.yaml: set analog: true, fill in top_module, title, author. Commit, wait for green, then submit at app.tinytapeout.com.

Important: analog tiles get a manual review in addition to the automated checks, so submit early. Don't wait until May 9.

Slide 47 Key Deadlines

Wrap. Key dates: April 17 is the beginner workshop — if you want to come help, please do. May 1 is the IEEE form. May 5 is token distribution. May 9 is the final deadline — after that the shuttle closes.

You just went from Verilog synthesis to hand-crafted analog layout. Those are the two halves of every mixed-signal ASIC, and the connection between them is this: a standard cell is just a transistor circuit that someone else built and characterized. Now you know how to build one yourself. Thank you for coming. TAs are here for questions.

Slide 48 XSCHEM Shortcuts Reference

Reference slide — leave on screen during the practical sections. Most-used: Insert (place), W (wire), Q (edit properties), Ctrl+S (save), Ctrl+Shift+S (save as).

TA Session

Pre-workshop briefing

Slide 1 Tiny Tapeout: TA Session

Welcome. This is a quick session for the TAs and helpers — the people who will be in the room circulating during the beginner and advanced workshops. The goal of this session is twofold: (1) get you set up with the analog tools yourself so you can help when participants get stuck, and (2) align on how we're using the tiles IEEE gave us.

Take 30 seconds at the door for introductions — name, year, what you've already touched (digital, analog, neither). Then move on.

Slide 2 While I Talk — Get xschem Running

Action item. Get xschem running on every TA's machine before we get to the analog content. The clone takes a few minutes and the Docker pull is large. Tell people to start the install *now* while you talk through the next two slides — introductions, plan, and the no-analog rule.

Common stumbles to flag up front: Mac users may need XQuartz; Windows users use the VNC fallback (`./start_vnc.sh`, browser to localhost:80); old Mac OS needs colima instead of Docker Desktop.

Slide 3 About Me

Same About Me pitch as the workshops. Quick: I'm Jack, ECE PhD student, VAK lab with Professor Arora. Ultra-low-power systems — battery-free sensors, energy-autonomous IoT. The reason I run this workshop is that I came to chip design as a systems person, not a VLSI person, and Tiny Tapeout is what made it accessible. The goal of these sessions is to make the same path easier for the next person.

Slide 4 What This Lab Is / Is Not

Same expectation-setting slide I give the workshop participants. I'm telling you so you can repeat it when you're circulating: this is hands-on, it's a bridge from the concepts they already know to the open-source tooling, and it is *not* a full VLSI course. The tools are not yet up to par with commercial EDA — which means there's room for them to contribute later. Nobody is being graded.

Slide 5 The Plan

This is the most important slide of the TA session. **IEEE Division 1 is giving us a few free tiles. Two of those are coming to this course.** That's it — two tiles total for the whole workshop.

The schedule: beginner session brings everyone up to speed on digital (Wokwi/Verilog). Advanced session brings everyone up to speed on the analog tools. Then everyone has until the hackathon to

flesh out their design. At the hackathon: everyone pitches their idea, and the group votes on which two designs actually go to tapeout.

Make sure the TAs understand: not everyone gets a tile. The vote at the hackathon is real, and it's on the merits.

Slide 6 Two Tiles, Not Two Designs

Two clarifications people always ask about. First: **2 tiles does not mean 2 separate designs**. We could do one large advanced digital project plus one beginner Wokwi project. Or two medium digital projects. The tiles are ours to use however the group decides at the hackathon.

Second: **the rule is no analog this round.** Why? The xschem learning curve is steep and I don't want anyone hung up on it for their first tapeout. We get a rep in with digital end-to-end first, then go deep.

Good news: we are launching a student club around this work. Analog will be the first project of the club. So if you came here for analog — you're in the right room, just on a slower timeline.